



User Guide

Getting Started with Linux Pre-Built Images

version: 0.5.2

Table of Contents

1. Introduction	4
2. Host Requirements	4
2.1 Hardware requirements.....	4
2.2 Software requirements	4
3. Hardware and Software Setup	4
3.1 Board Setup:.....	4
Install Alif Security Toolkit (SETOOLS).....	5
Connecting USB PRG Cable for Getting Serial Consoles	5
UART4 Connection for GDB Debug Over Serial Cable.....	6
3.2 Downloading Linux Beta Distribution 0.5.2 Files	7
3.3 Extracting the Release and Copying Pre-Built Images and Configuration Files	7
4 Programming Prebuilt Images	7
4.1 Programming Linux Images for MRAM Operation:	7
4.2 Programming Linux Images For OSPI1 flash XIP Operation:	8
4.3 Further Instructions	9
5 Application Development	10
5.1 Creating A Partitioned SD Card For Program Transfer.....	10
Insert an SD card into Linux host.	10
Create ext4 partition using following commands.....	10
Format the first partitions.....	12
5.2 Cross-Compile User Application, Save to SD Card, and Execute.....	13
6 Debugging An Application with GDB Over Serial Cable	17
6.1 Overview	17
6.2 Prerequisites	18
6.3 GDB client-server debug on Linux host.....	19
6.4 Limitations.....	22
7 List Of Linux Kernel Drivers Supported	23
7.1 MHU (Message Handling Unit)	23
7.2 HWSEM (Hardware Semaphore)	23
7.3 Watchdog (SBASA Generic Watchdog timer).....	24
7.4 USB Device mode (DWC3 Dual role controller)	26
7.5 I2C ENV3-CLICK Temperature Sensor	28
7.6 SPI ENV3-CLICK Temperature Sensor.....	29

7.7 USB Device Detection	31
7.8 CDC-200.....	31
7.9 CRC	32
7.10 D/AVE-2D	33
8 Linux device drivers RAM usage	34
9 Linux Kernel Feature	35
10 Reference	36
11 Document History.....	37

1. Introduction

The Alif DevKit is a development device containing an Alif Ensemble multi-core high-performance, low-power consumption embedded device. It consists of multiple Cortex-M class and Cortex-A class processors on a single device. The Cortex-A32 processor present in the device is a low-power 32-bit ARMv8-A only processor that offers ultra-high efficiency in computing. The Application Processor Sub System (APSS) is made up of one or more Cortex-A32 cores, which can be used to run high-level/rich operating systems like Linux.

The APSS in the DevKit Ensemble device has the following features,

- CPU core(s) operating at 800MHz.
- CPU core(s) each with L1 instruction cache of 32KB, L1 data cache of 32 KB.
- Total L2 cache of 512KB.
- SRAM (static random-access memory) of 4MB.
- MRAM (Magnetoresistive Random Access Memory) of 6MB
- Advanced SIMD and floating-point support.
- GIC-400 (Generic Interrupt Controller) support.
- Supports execution state at all Exception levels (EL0 to EL3).
- Performance Monitoring Unit (PMU).
- Embedded Trace Macrocell (ETM).

In this User Guide, we will focus on downloading the installing the pre-built images necessary to run the beta version of the Alif Linux distribution on the Ensemble DevKit using OSPI flash and OSPI HyperRAM.

A list of the drivers supported in this pre-built release image is available in section 7.

2. Host Requirements

2.1 Hardware requirements

1. Personal Computer (PC) with x86-64 based Processor with minimum 4GB RAM and disk space of at least 256 GB.
2. Alif Ensemble DevKit (DK-E7) Development Kit

2.2 Software requirements

1. PC running any of the following OS.
 - Windows 10 or above 64-bit OS.
 - 64-bit Linux distribution Ubuntu 18.04 or higher, Cent OS 7.x/Fedora 28 or higher.
2. Alif Security Toolkit version v1.94.0 or later

3. Hardware and Software Setup

3.1 Board Setup:

You should have set up your Ensemble DevKit according to the [Ensemble DevKit User Guide \(Gen 2\)](#) and verifying that the factory blinky program runs correctly.

Install Alif Security Toolkit (SETOOLS)

Install the latest copy of the Alif Security Toolkit (SETOOLS) as per the [Alif Security Toolkit Quick Start Guide](#). Go to the Alif website and download the latest version of the Alif Security Toolkit. The linux version is at <https://alifsemi.com/download/APFW0002> . Install it on your Linux machine. The default location is at `/home/$USER/app-release-exec-linux`

Connecting USB PRG Cable for Getting Serial Consoles

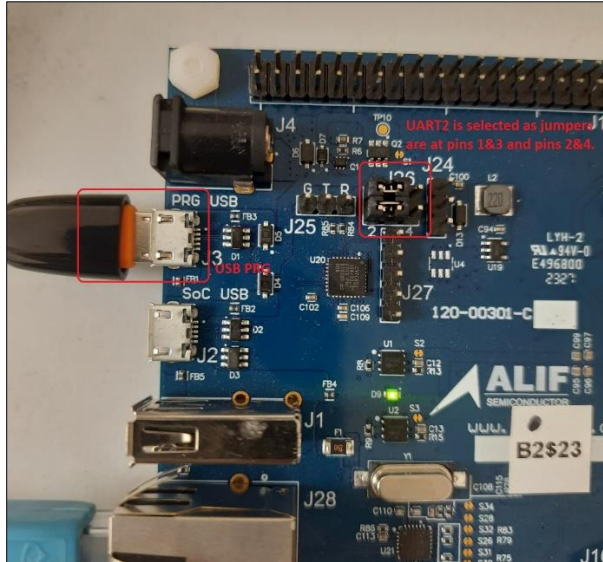


Figure 1: Connecting the USB PRG Cable and Configuring Jumper Connections for UART2

The USB PRG J3 port provides two available ports:

- SEUART port.
- UART2 or UART4 port

To choose between UART2 and UART4 ports, adjust the J26 jumper settings, as shown in the picture below. If the jumpers are connected to pins 1 and 3 and pins 2 and 4, then the UART2 port is selected as shown in the picture above to enable viewing Linux console output.

Note that the boards are shipped from the factory set up for UART4 output so this has to be changed to run Linux and view the console.

Note:

In this user guide, we will use UART 2 as the serial console for APSS. Therefore, all APSS boot-up messages should be visible on UART2.

The UART4 is used for debugging user-space applications using the GDBserver over serial cable. Therefore, the following hardware connection is required for GDB debugging.

UART4 Connection for GDB Debug Over Serial Cable

IMPORTANT:



DO THIS BEFORE APPLYING POWER TO THE BOARD OR PLUGGING THE FTDI MODULE INTO A USB PORT:

Set the jumper on the FTDI module for 1.8V TTL levels as shown in the image on the right. This must be done before plugging the FTDI module in a USB port or damage may result to the CPU board. The jumper must be on the pair of pins closest to the USB connector.

Note:

Variation in the colors of cables supplied with the adapter may occur. The examples below use a cable connected to the adapter as shown in the image below. Please verify the color of the wires connected to various FTDI signals, as identified by the signal names silkscreened on the adapter PCB.

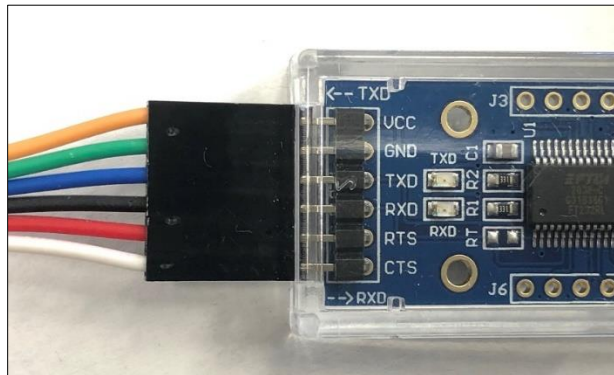


Figure 2: FTDI Connection Pin for UART4

Connect the FTDI UART cable between the host PC and the Devkit device UART4 connection using the following UART4 pins:

- J26 pin 6 (UART4_RX): Connects to TXDATA (blue wire or as noted above) on the FTDI cable.
- J26 pin 5 (UART4_TX): Connects to RXDATA (black wire or as noted above) on the FTDI cable.
- J27 pin 1 (GND): Connects to GND (green wire or as noted above) on the FTDI cable.

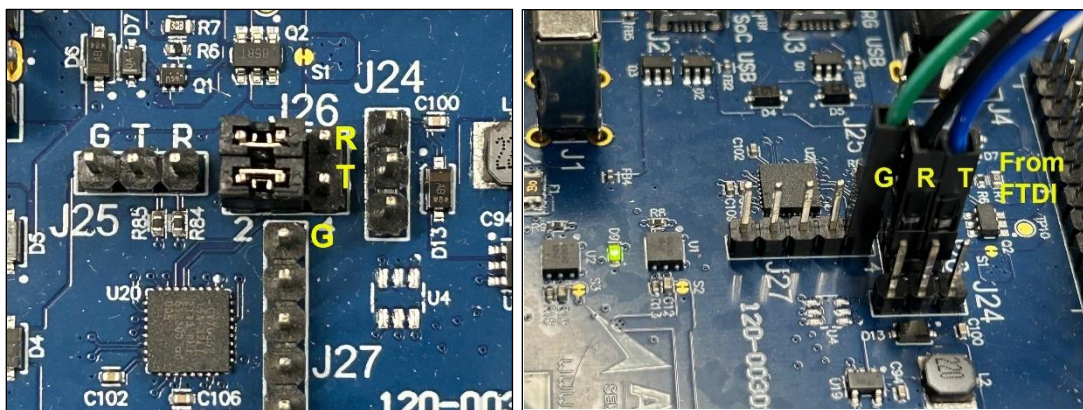


Figure 3: UART4 Pin Connections for Ensemble DevKit

3.2 Downloading Linux Beta Distribution 0.5.2 Files

Save the Linux v0.5.2 Beta distribution zip file to your */home/\$USER* directory.

Extract the contents of the zip file to a directory of your choice.

By default, the contents will be saved under *./APSS-v0.5.2-Beta*.

3.3 Extracting the Release and Copying Pre-Built Images and Configuration Files

Loading Prebuilt Alif Linux Images

Copy the prebuilt images and config files to the appropriate SETOOLS directory.

```
$ cd /home/$USER/alif_linux-0.5.2/config-files
$ cp *.* /home/$USER/app-release-exec-linux/build/config
$ cd /home/$USER/alif_linux-0.5.2/prebuilt-images
$ cp *.* /home/$USER/app-release-exec-linux/build/images
```

There are two sets of pre-built images available.

MRAM Operation:

This set of images stores all four Linux images (bl32, device tree, kernel, and file system) in the on-chip MRAM. On-chip SRAM is used for kernel memory. The advantage of this distribution is that it can be programmed using a single SETOOLS command. This distribution supports SD-share, USB, I2C, SPI, CRC, and GDB debug using UART4.

OSPI Flash Operation:

This set of images stores the boot loader (bl32) and device tree in on-chip MRAM but stores the larger kernel and file system images in the off-chip OSPI flash memory. This uses much less of the on-chip MRAM so it is available for M55 real-time applications but requires multiple steps to load both the off-chip OSPI flash and the on-chip MRAM. This distribution supports the peripherals in the MRAM distribution plus Ethernet.

4 Programming Prebuilt Images

4.1 Programming Linux Images for MRAM Operation:

```
$ cd /home/$USER/app-release-exec-linux
$ ./app-gen-toc -f build/config/APSS-B2-MRAM.json
$ sudo ./app-write-mram -p
```

When programming is done, the board will boot Linux with console output on UART2 (*/dev/ttyACM1*). Press RETURN within 5 seconds to get a login prompt. Log in with “root” as the username.

Note:

The *app-write-mram* option “-p” adds additional bytes to the binaries if necessary to make them 16-byte aligned.

4.2 Programming Linux Images For OSPI1 flash XIP Operation:

Program MRAM with the PC flash burner M55_HP application

```
$ cd /home/$USER/app-release-exec-linux
$ ./app-gen-toc -f build/config/APSS-B2-PC-tool-burner.json
$ sudo ./app-write-mram -p
```

Run the PC flashtool:

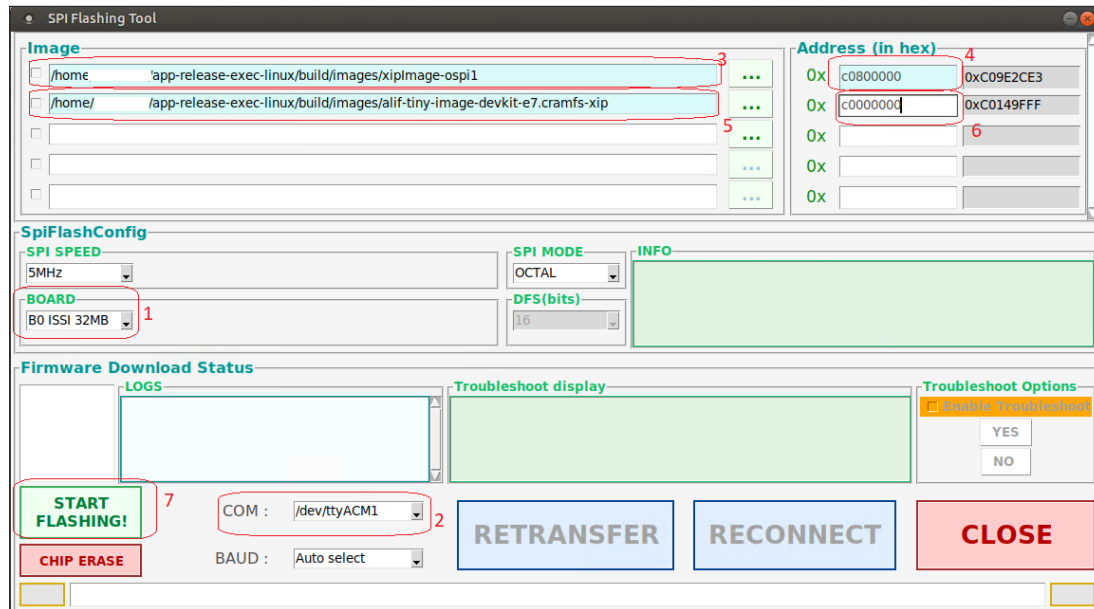
```
$ cd /home/$USER/ alif_linux-0.5.2/Utilities/PC-Tool/
$ sudo ./flashtool
```

In the flashtool, set parameters as follows:

```
BOARD = "B0 ISSI 32MB"
COM = "/dev/ttyACM1"
IMAGE = <path-to-prebuilt-images>/xipimage-osp1
ADDRESS = 0xC0800000
IMAGE = <path-to-prebuilt-images>/alif-tiny-image-devkit-e7.cramfs-xip
ADDRESS = 0xC0000000
```

Press RESET on the DevKit board

Press "START FLASHING"



When the tool finishes, press "CLOSE"

Program MRAM with the boot loader and device tree:

```
$ cd /home/$USER/app-release-exec-linux
$ ./app-gen-toc -f build/config/APSS-B2-OSPI1.json
$ sudo ./app-write-mram -p
```

When programming is done, the board will boot Linux with console output on UART2 (/dev/ttyACM1).

Press RETURN within 5 seconds to get a login prompt. Log in with "root" as the username.

4.3 Further Instructions

Setting Up the Environment for Building Custom Images

In order to move beyond the prebuilt images and customize a Linux distribution for your system, you will need to set up the environment for building images with different system configurations.

Go to this repo the Alif Semiconductor GitHub: https://github.com/alifsemi/alif_linux-apss-build-setup and follow the instructions in the README file.

If you have any questions on the process, please file a support request ticket at <https://alifsemi.com/support/technical-support/>

5 Application Development

5.1 Creating A Partitioned SD Card For Program Transfer

This section has details on how to create a formatted SD card that can be used to transfer the compiled program from the host system to the DevKit for execution and debugging. If you already have a formatted card, you can skip to section 5.2.

Insert an SD card into Linux host.

- A. The inserted SD card usually detects as `/dev/sdX` device node on Linux host or Linux VMware running on Windows 10. The same device node can be accessed inside the Linux host or Linux VMware running on Windows 10. Check the SD device node using “`dmesg`” or “`sudo fdisk -l`” command.

Example: The inserted SD card detects as `/dev/sdc`.

Create ext4 partition using following commands.

Note: Replace `/dev/sdc` with the actual SD card device node detected at your setup.

Command: `sudo fdisk /dev/sdc`

- I. Use `p` to print the current partition table to the terminal from within command mode.
 - Command (m for help): `p`
- II. Use the `d` command to delete a partition if it there in SD CARD.
 - Command (m for help): `d`
- III. Use the `n` command to create a new partition.
 - Command (m for help): `n`
- IV. Use the `n` command to create a 1st new 2GB ext4 partition.
 - Command (m for help): `n`
 - Partition number (1-128, default 1): `<Press Enter>`
 - First sector (2048-50011815, default 2048): `<Press Enter>`
 - Last sector, +/-sectors or +/-size{K,M,G,T,P} (2048-500118158, default 500118158): `<+2G>`
- V. Save the changes by running the `w` command:
 - Command (m for help): `w`
- VI. Type ‘`q`’ to come out of `fdisk` prompt
 - Command (m for help): `q`

```
$ sudo fdisk /dev/sdc

Welcome to fdisk (util-linux 2.31.1).
Changes will remain in memory only, until you decide to write them.
Be careful before using the write command.


Command (m for help): p
Disk /dev/sdc: 29.8 GiB, 31976325120 bytes, 62453760 sectors
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disklabel type: dos
Disk identifier: 0x5cae097e


Device      Boot Start      End Sectors Size Id Type
/dev/sdc1                2048 4196351 4194304   2G 83 Linux


Command (m for help): d
Selected partition 1
Partition 1 has been deleted.


Command (m for help): n
Partition type
  p   primary (0 primary, 0 extended, 4 free)
  e   extended (container for logical partitions)
Select (default p): p
Partition number (1-4, default 1): 1
First sector (2048-62453759, default 2048):
Last sector, +sectors or +size{K,M,G,T,P} (2048-62453759, default 62453759): +2G

Created a new partition 1 of type 'Linux' and of size 2 GiB.
Partition #1 contains a ext4 signature.

Do you want to remove the signature? [Y]es/[N]o: y

The signature will be removed by a write command.


Command (m for help): w
The partition table has been altered.
Calling ioctl() to re-read partition table.
Syncing disks.


$ sudo mkfs.ext4 /dev/sdc1
mkfs.ext4 1.44.1 (24-Mar-2018)
Creating filesystem with 524288 4k blocks and 131072 inodes
Filesystem UUID: 64bd55aa-80e3-4cfb-aec1-8b16639f4d11
Superblock backups stored on blocks:
    32768, 98304, 163840, 229376, 294912

Allocating group tables: done
```

Format the first partitions.

Note: Replace /dev/sdc with the actual SD card device node detected at your setup.

Command: `sudo mkfs.ext4 /dev/sdc1`

```
$ sudo mkfs.ext4 /dev/sdc1
mke2fs 1.44.1 (24-Mar-2018)
Creating filesystem with 524288 4k blocks and 131072 inodes
Filesystem UUID: 64bd55aa-80e3-4cfb-aec1-8b16639f4d11
Superblock backups stored on blocks:
    32768, 98304, 163840, 229376, 294912

Allocating group tables: done
Writing inode tables: done
Creating journal (16384 blocks): done
Writing superblocks and filesystem accounting information: done
```

5.2 Cross-Compile User Application, Save to SD Card, and Execute

The following steps show how to cross-compile a userspace embedded application to be debugged and how to transfer it to the development kit using an ext4 partition of an SD card.

1. Open a new bash shell as a normal user on Linux distribution running on a host machine or inside Linux VMware running on Windows 10 and go to the “SDK” sub-directory of the beta release.
\$ cd ~/APSS-v0.5.2-Beta/SDK
2. Install the shipped toolchain installer. By default, the toolchain installs at /opt/apss-tiny/0.1 directory and it can be changed when the installer asks for the “Enter target directory for SDK” query.

```
$ sh apss-tiny-musl-x86_64-alif-tiny-image-cortexa32hf-neon-devkit-e7-toolchain-0.1.sh
```

```

$ sh apss-tiny-musl-x86_64-alif-tiny-image-cortexa32hf-neon-devkit-e7-toolchain-0.1.sh
Alif Iota - Tiny Linux Distribution SDK installer version 0.1
=====
Enter target directory for SDK (default: /opt/apss-tiny/0.1):
The directory "/opt/apss-tiny/0.1" already contains a SDK for this architecture.
If you continue, existing files will be overwritten! Proceed [y/N]? y
Extracting SDK.....done
Setting it up...done
SDK has been successfully set up and is ready to be used.
Each time you wish to use the SDK in a new shell session, you need to source the environment setup script e.g.
$ . /opt/apss-tiny/0.1/environment-setup-cortexa32hf-neon-poky-linux-musleabi
$ source /opt/apss-tiny/0.1/environment-setup-cortexa32hf-neon-poky-linux-musleabi

```

3. Source the environment-setup-cortexa32hf-neon-poky-linux-musleabi script.

```
$ source /opt/apss-tiny/0.1/environment-setup-cortexa32hf-neon-poky-linux-musleabi
```

Example:

```
$ source /opt/apss-tiny/0.1/environment-setup-cortexa32hf-neon-poky-linux-musleabi
```

Any time you open a new terminal window to cross-compile an application, you need to source this environment setup file.

4. Create a projects directory for your user application and change directory to it. In these examples, we have used ~/projects.
5. Create a sample embedded application file (helloworld.c) as below,

```

$ cat helloworld.c
#include <stdio.h>
#include <unistd.h>
int main() {
    for(;;) {
        printf("Linux Hello World Example 1\n");
        sleep(1);
    }
}

```

```
$ cat helloworld.c
#include <stdio.h>
#include <unistd.h>

int main() {
    for(;;) {
        printf("Linux Hello World Example 1\n");
        sleep(1);
    }
    return 0;
}
```

Note: This source file is also supplied in the APSS-v0.5.1-Beta/SDK directory.

6. GCC is used to compile the embedded application code with “-g -O0” flags which helps to get useful information while debug.

```
$ echo $CC
```

```
arm-poky-linux-musleabi-gcc -mcpu=cortex-a32 -march=armv8-a+crc -mfp=neon -mfloat-abi=hard -fstack-protector-strong -D_FORTIFY_SOURCE=2 -Wformat -Wformat-security -Werror=format-security --sysroot=/var/lib/jenkins/DevKit_E7_APSS_Linux_Project/build-scripts/opt/apss-tiny/0.1/sysroots/cortexa32hf-neon-poky-linux-musleabi
```

```
$ $CC helloworld.c -g -O0 -o helloworld
```

```
$ echo $CC
arm-poky-linux-musleabi-gcc -mcpu=cortex-a32 -march=armv8-a+crc -mfp=neon -mfloat-abi=hard -fstack-protector-strong -D_FORTIFY_SOURCE=2 -Wformat -Wformat-security -Werror=format-security --sysroot=/home/ /apss-tiny/0.1/sysroots/cortexa32hf-neon-poky-linux-musleabi
$ gcc helloworld.c -g -O0 -o helloworld
$ cp helloworld helloworld_unstripped
$ arm-poky-linux-musleabi-strip helloworld
```

7. Create a backup of the unstripped helloworld executable file.
\$ cp helloworld helloworld_unstripped
8. Remove debug symbols from the helloworld executable file.
\$ arm-poky-linux-musleabi-strip helloworld
9. Insert a partitioned SD card into Linux host. If you need instructions on creating a partitioned SD card, refer to section 5 above.

10. Follow below steps to copy embedded Linux application (helloworld) to be debugged into an ext4 partition of the SD card.

- A. Insert a partitioned SD card into Linux host.
- B. The inserted SD card usually detects as /dev/sdX device node on Linux host or Linux VMware running on Windows 10. The same device node can be accessed inside the Linux host or Linux VMware running on Windows 10. Check the SD device node using “dmesg” or “sudo fdisk -l” command.

Example: The inserted SD card detects as /dev/sdc.

- C. Mount the first partition of SD card.

Note: Replace /dev/sdc with the actual SD card device node detected at your setup.

```
$ mkdir -p ~/sd_card_mnt
```

```
$ sudo mount /dev/sdc1 ~/sd_card_mnt
```

```
$ mkdir ~/sd_card_mnt
```

```
$ sudo mount /dev/sdc1 ~/sd_card_mnt/
```

- D. Copy the embedded application executable (helloworld), which needs to be debugged generated at step 5 to ~/sd_card_mnt directory.

```
$ sudo cp helloworld ~/sd_card_mnt/
```

In order to make the file copied to the SD card readable on the DevKit, you can change the permission modes by running

```
$ sudo chmod a+x ~/sd_card_mnt/helloworld
```

```
$ sudo cp helloworld ~/sd_card_mnt/
$ ls -lh ~/sd_card_mnt/
total 8.0K
-rw-r--r-- 1 root root 5.3K Jul 18 13:51 helloworld
$ sudo chmod a+x ~/sd_card_mnt/helloworld
$ ls -lh ~/sd_card_mnt/
total 8.0K
-rwxr-xr-x 1 root root 5.3K Jul 18 13:51 helloworld
$
```

- E. Unmount the ~/sd_card_mnt directory.

```
$ sudo umount ~/sd_card_mnt
```

```
$ sudo umount ~/sd_card_mnt
$
```

11. Remove the SD card from the host.
12. Insert the SD card into Devkit device.
13. Power on the DevKit, open a remote terminal window on your UART2 interface at 115200 bps and log in as "root".
14. Create a new directory (say sd_share) under /var/volatile directory to mount the SD card inserted in the Devkit device. The SD card usually detects as /dev/mmcblk0px, where "x" can be 0,1, etc. Please see Linux boot up message to find out correct SD card.

Command: mkdir /var/volatile/sd_share

```
root@devkit-e7:~# stty -F /dev/ttyS2 115200 -brkint -icrnl -imaxbel -opost -onlcr -isig -icanon -texten -echo -echoe -echok -echoctl -echoke
root@devkit-e7:~# mkdir /var/volatile/sd_share
root@devkit-e7:~# mount /dev/mmcblk0p1 /var/volatile/sd_share
EXT4-fs (mmcblk0p1): mounted filesystem with ordered data mode. Opts: (null)
root@devkit-e7:~#
```

15. Boot up log to find out detected SD card:


```
sbsa-gwdt 1a300000.watchdog: Initialized with 100s timeout @ 100000000 Hz, action=0.
sdhci: Secure Digital Host Controller Interface driver
sdhci: Copyright(c) Pierre Ossman
sdhci-pltfm: SDHCI platform and OF driver helper
mmc0: SDHCI controller on 48102000.sdhci [48102000.sdhci] using ADMA
mhuv2 1b000000.mhu: ARM MHUV2 Mailbox driver registered
mhuv2 1b020000.mhu: ARM MHUV2 Mailbox driver registered
mhuv2 1b040000.mhu: ARM MHUV2 Mailbox driver registered
mhuv2 1b060000.mhu: ARM MHUV2 Mailbox driver registered
mhuv2 1b800000.mhu: ARM MHUV2 Mailbox driver registered
mhuv2 1b820000.mhu: ARM MHUV2 Mailbox driver registered
devkit-rtc 70010000.rtc: setting system clock to 1970-01-01T01:31:43 UTC (5503)
ttyS1 - failed to request DMA
cramfs: checking physical address 0xd0400000 for linear cramfs image
cramfs: linear cramfs image on mtd:physmap-flash.0 appears to be 3276 KB in size
VFS: Mounted root (cramfs filesystem) readonly on device 31:0.
Freeing unused kernel memory: 24K
This architecture does not have kernel memory protection.
Run /sbin/init as init process
mount: mounting sysfs on /sys failed: No such device
ttyS0 - failed to request DMA

Alif Iota - Tiny Linux Distribution 0.1 devkit-e7 /dev/ttyS1

devkit-e7 login: mmc0: new high speed SDHC card at address aaaa
mmcblk0: mmc0:aaaa SD32G 29.7 GiB
mmcblk0: p1

Alif Iota - Tiny Linux Distribution 0.1 devkit-e7 /dev/ttyS1
```

16. Mount the detected SD card using mount command.

Command: `mount /dev/mmcblk0p1 /var/volatile/sd_share`

```
root@devkit-e7:~# stty -F /dev/ttyS2 115200 -brkint -icrnl -imaxbel -opost -onlcr -isig -icanon -iexten -echo -echoe -echok -echoctl -echoke
root@devkit-e7:~# mkdir /var/volatile/sd_share
root@devkit-e7:~# mount /dev/mmcblk0p1 /var/volatile/sd_share
EXT4-fs (mmcblk0p1): mounted filesystem with ordered data mode. Opts: (null)
root@devkit-e7:~#
```

17. Run the program by going to your project directory on the SD card. (/var/volatile/sd_share for this example) and enter “./helloworld”. You should see the sample output on your console window.

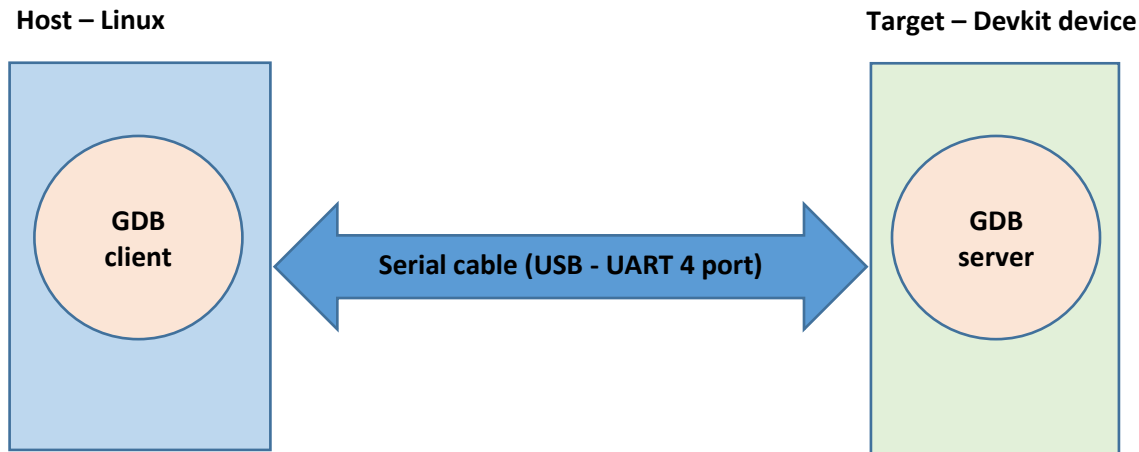
6 Debugging An Application with GDB Over Serial Cable

6.1 Overview

Linux userspace embedded applications are usually debugged using GNU Debugger (GDB), a portable debugger that runs on many Unix-like systems. The same method is used in this user guide to debug userspace embedded applications running on the APSS.

The GDBserver tool of GDB is a computer program that makes it possible to remotely debug the Linux userspace applications. The GDBserver runs on the target system (DevKit Ex) as the applications to be debugged, it allows the client GDB tool to connect from another system (host machine either Windows 10 or Linux in our case); that is, only the application to be debugged needs to be resident on the target system (Devkit Ex), while the source code and a copy of the binary file (with debug symbols) to be debugged reside on the developer's local computer ("host"). The connection can be either ethernet or a serial line.

The following figure shows schematic view of how remote debug using GDB client-server via serial (UART4 port) works.



1. **GDBserver** is launched on the Devkit device, with the arguments:
 - a. The serial device name (e.g.: /dev/ttyS2) of the UART4 port.
 - b. The path and filename of the executable to be debugged.
 It then waits passively for the host gdb to communicate with it.
2. **GDB** is run on either Linux or Windows 10 host, with the arguments:
 - a. The path and filename of the executable (and any sources) on the host, and
 - b. The USB device name (e.g: /dev/ttyUSBx for Linux Host) of USB port, which is used to connect with the Devkit device.

6.2 Prerequisites

To debug userspace embedded applications using a GDB client- server model using serial cable which is connected to UART4 of the development kit at one end and to a USB port of the Linux host at other end, it is necessary to use a distribution with serial (UART4) support so that the Devkit device communicates with the host machine and vice-versa. The embedded Linux application to be debugged is copied into an ext4 partition of an SD card which is mounted within the Linux rootfs.

The pre-built images that were programmed into the development kit in the previous section have these features enabled and can be used to do GDB client-server debugging of user applications.

Because the Ensemble DevKit Gen 2 only has the SEUART and one of the UART ports available on the micro-USB connector, the normal setup for Linux is to have the jumpers set to have UART2 appear on the micro-USB connection for use as the console.

An external FTDI adapter needs to be hooked up to get simultaneous access to UART4 for debugging at the same time as UART2 for the Linux console. The instructions for connecting an external FTDI adapter to UART4 are in section 3 of this document.

In order to determine which ttys port is being used for UART4, it is suggested that you check the dmesg file with the following command:

```
root@devkit-e7:/var/volatile/Symetrica# dmesg |grep tty
Kernel command line: console=ttyS0,115200n8      root=mtd:physmap-
flash.0      rootfstype=cramfs      ro      loglevel=9
printk: console [ttyS0] disabled
4901a000.uart2: ttyS0 at MMIO 0x4901a000 (irq = 128, base_baud = 6250000) is a 16550A
printk: console [ttyS0] enabled
4901c000.uart4: ttyS1 at MMIO 0x4901c000 (irq = 129, base_baud = 6250000) is a 16550A
```

In this case, it is obvious that UART4 is on ttyS1.

6.3 GDB client-server debug on Linux host

At Devkit device:

1. Run the following command to set baud rate for the UART4 port (e.g: /dev/ttyS2).
Command: `stty -F /dev/ttyS2 115200 -brkint -icrnl -imaxbel -opost -onlcr -isig -icanon -iexten -echo -echoe -echok -echoctl -echoke`

```
stty -F /dev/ttyS1 115200 -brkint -icrnl -imaxbel -opost -onlcr -isig -icanon -iexten -echo -echoe -echok -echoctl -echoke
```

```
root@devkit-e7:~# stty -F /dev/ttyS2 115200 -brkint -icrnl -imaxbel -opost -onlcr -isig -icanon -iexten -echo -echoe -echok -echoctl -echoke
root@devkit-e7:~# mkdir /var/volatile/sd_share
root@devkit-e7:~# mount /dev/mmcblk0p1 /var/volatile/sd_share
EXT4-fs (mmcblk0p1): mounted filesystem with ordered data mode. Opts: (null)
root@devkit-e7:~#
```

2. Launch gdbserver with serial device name of the UART4 port (e.g.: /dev/ttyS2) and userspace embedded application (example: helloworld) to be debugged on the Devkit device.

```
Command: gdbserver /dev/ttyS2 /var/volatile/sd_share/helloworld
root@devkit-e7:~# gdbserver /dev/ttyS2 /var/volatile/sd_share/helloworld
Process /var/volatile/sd_share/helloworld created; pid = 56
Remote debugging using /dev/ttyS2
```

3. The gdbserver waits for the GDB client from the host machine to connect.

At Linux host:

4. Open a terminal window in your chosen projects directory. In these examples, we have used ~/projects
5. Assuming you have already installed the SDK as detailed in the previous sections, source the environment-setup-cortexa32hf-neon-poky-linux-musleabi script. In this example, the SDK is installed at /opt/apss-tiny/0.1

```
$ source ./environment-setup-cortexa32hf-neon-poky-linux-musleabi (from SDK directory)
```

Example:

```
$ source /opt/apss-tiny/0.1/environment-setup-cortexa32hf-neon-poky-linux-musleabi
```

```
$ sh apss-tiny-musl-x86_64-alif-tiny-image-cortexa32hf-neon-devkit-e7-toolchain-0.1.sh
Alif Iota - Tiny Linux Distribution SDK installer version 0.1
=====
Enter target directory for SDK (default: /opt/apss-tiny/0.1):
The directory "/opt/apss-tiny/0.1" already contains a SDK for this architecture.
If you continue, existing files will be overwritten! Proceed [y/N]? y
Extracting SDK.....done
Setting it up...done
SDK has been successfully set up and is ready to be used.
Each time you wish to use the SDK in a new shell session, you need to source the environment setup script e.g.
$ . /opt/apss-tiny/0.1/environment-setup-cortexa32hf-neon-poky-linux-musleabi
$ source /opt/apss-tiny/0.1/environment-setup-cortexa32hf-neon-poky-linux-musleabi
```

6. Run cross-gdb (arm-poky-linux-musleabi-gdb) as super user using 'sudo' along with unstripped Linux userspace application (helloworld_unstripped) inside the Linux host or Linux VMware running on Windows 10.

Example:

```
$ sudo arm-poky-linux-musleabi-gdb /PATH/TO/helloworld_unstripped
$ sudo arm-poky-linux-musleabi-gdb helloworld_unstripped
GNU gdb (GDB) 8.3.1
Copyright (C) 2019 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "--host=x86_64-pokysdk-linux --target=arm-poky-linux-gnueabi".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<http://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
    <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from /home/hemanth/Downloads/helloworld_unstripped...
(gdb) set serial baud 115200
(gdb) target remote /dev/ttyUSB2
```

7. Inside the GDB prompt, set serial baud-rate to 115200. Connect to remote target via serial using “target remote <USB port having FTDI serial cable’s USB plug>” command.
Replace,
 - USB port having FTDI serial cable’s USB plug – One end of FTDI serial cable is connected to USB port of Linux host ((e.g: /dev/ttyUSB3) and the other end is connected to UART 4 of Devkit device.

```
(gdb) set serial baud 115200
(gdb) target remote /dev/ttyUSBx
```

Note: Replace ‘x’ with the actual USB port number.

```
$ sudo arm-poky-linux-musleabi-gdb helloworld_unstripped
GNU gdb (GDB) 8.3.1
Copyright (C) 2019 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "--host=x86_64-pokysdk-linux --target=arm-poky-linux-gnueabi".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<http://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
    <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from /home/hemanth/Downloads/helloworld_unstripped...
(gdb) set serial baud 115200
(gdb) target remote /dev/ttyUSB2
```

8. After successful connection, the following messages are displayed.

```
(gdb) set serial baud 115200
(gdb) target remote /dev/ttyUSB2
Remote debugging using /dev/ttyUSB2
Reading /lib/ld-linux-armhf.so.3 from remote target...
Warning: File transfers from remote targets can be slow. Use "set sysroot" to access files locally instead.
Reading /lib/ld-linux-armhf.so.3 from remote target...
Reading symbols from target:/lib/ld-linux-armhf.so.3...
Reading /lib/libc.so from remote target...
Reading /lib/.debug/libc.so from remote target...
Reading /opt/apss-tiny/0.1/sysroots/x86_64-pokysdk-linux/usr/lib/arm-poky-linux-gnueabi/debug//lib/libc.so from remote target...
Reading /opt/apss-tiny/0.1/sysroots/x86_64-pokysdk-linux/usr/lib/arm-poky-linux-gnueabi/debug/lib/libc.so from remote target...
Reading target:/opt/apss-tiny/0.1/sysroots/x86_64-pokysdk-linux/usr/lib/arm-poky-linux-gnueabi/debug/lib/libc.so from remote target...
(No debugging symbols found in target:/lib/ld-linux-armhf.so.3)
0xb6fb7cf0 in _dlstart () from target:/lib/ld-linux-armhf.so.3
(gdb)
```

9. Type the following within the GDB prompt. The GDB stops at main function.

```
(gdb) b main
(gdb) c
Reading /opt/apss-tiny/0.1/sysroots/x86_64-pokysdk-linux/usr/lib/arm-poky-linux-gnueabi/debug//lib/libc.so from remote target...
Reading /opt/apss-tiny/0.1/sysroots/x86_64-pokysdk-linux/usr/lib/arm-poky-linux-gnueabi/debug/lib/libc.so from remote target...
Reading target:/opt/apss-tiny/0.1/sysroots/x86_64-pokysdk-linux/usr/lib/arm-poky-linux-gnueabi/debug/lib/libc.so from remote target...
(No debugging symbols found in target:/lib/ld-linux-armhf.so.3)
0xb6fb7cf0 in _dlstart () from target:/lib/ld-linux-armhf.so.3
(gdb) b main
Breakpoint 1 at 0x4005f0: file helloworld.c, line 7.
(gdb) info b
Num Type Disp Enb Address What
1 breakpoint keep y 0x004005f0 in main at helloworld.c:7
(gdb) disas main
Dump of assembler code for function main:
0x004005e8 <+0>: push {r11, lr}
0x004005ec <+4>: add r11, sp, #4
0x004005f0 <+8>: ldr r3, [pc, #20] ; 0x40060c <main+36>
0x004005f4 <+12>: add r3, pc, r3
0x004005f8 <+16>: mov r0, r3
0x004005fc <+20>: bl 0x4003b8 <puts@plt>
0x00400600 <+24>: mov r0, #1
```

```
(gdb) i b
Num      Type      Disp Enb Address      What
1        breakpoint keep y  0x004005f0 in main at helloworld.c:7
          breakpoint already hit 2 times
2        breakpoint keep y  0x004005fc in main at helloworld.c:7
          breakpoint already hit 1 time
(gdb) del 1
(gdb) i b
Num      Type      Disp Enb Address      What
2        breakpoint keep y  0x004005fc in main at helloworld.c:7
          breakpoint already hit 1 time
(gdb) c
Continuing.

Breakpoint 2, 0x004005fc in main () at helloworld.c:7
7      printf("Linux Hello World Example 1\n");
(gdb) c
Continuing.

Breakpoint 2, 0x004005fc in main () at helloworld.c:7
7      printf("Linux Hello World Example 1\n");
(gdb) c
Continuing.

Breakpoint 2, 0x004005fc in main () at helloworld.c:7
7      printf("Linux Hello World Example 1\n");
(gdb) c
Continuing.

Breakpoint 2, 0x004005fc in main () at helloworld.c:7
7      printf("Linux Hello World Example 1\n");
(gdb) █
```

10. Type the various GDB commands within GDB prompt for debugging

- b <function> - put breakpoint at <function>
- n – single step over
- s – single step into
- b *<address> – put breakpoint at <address>
- list – view source code being debugged
- i r – to view register information
- i b – to view list of breakpoints
- del – to delete break points
- q – quit

11. Please refer to [GDB quick reference](#) to know more about GDB commands.

12. The output of the embedded application (helloworld) is displayed on console running the gdbserver at Devkit device.

```
root@devkit-e7:~# gdbserver /dev/ttyS2 /var/volatile/sd_share/helloworld
Process /var/volatile/sd_share/helloworld created; pid = 56
Remote debugging using /dev/ttyS2
random: crng init done
Linux Hello World Example 1
█
```

6.4 Limitations

1. After successful GDB client-server debugging the gdbserver does not exit gracefully on the Devkit device. Reset the Devkit device to start GDB client-server debugging again.

7 List Of Linux Kernel Drivers Supported

The following are the list of Linux drivers supported in this release,

1. MHU (Message Handling Unit)
2. HWSEM (Hardware Semaphore)
3. Watchdog (SBSA Generic Watchdog timer)
4. USB Device mode (DWC3 Dual role controller)
5. I2C Temperature Sensor (ENV3 Click)
6. SPI Temperature Sensor (ENV3 - Click)
7. Ethernet
8. CDC-200
9. CRC (Cyclic Redundancy Check)
10. D/AVE-2D
11. SD/MMC

7.1 MHU (Message Handling Unit)

MHU facilitates the communications between two independent cores using Linux Kernel's Mailbox and RPMSG driver on a Heterogeneous Multicore Processing (HMP) platform like DevKit.

An MHU has two halves. A Sender frame and a Receiver frame, sometimes referred to as Sender and Receiver. The MHU facilitates communication between two cores, with one core having the Sender and the other having the Receiver of a single MHU. MHUs are implemented in pairs to allow for full-duplex communications, with Sender and Receiver being reversed between the two MHUs in the pair.

On the APSS, the Linux Kernel xiplImage can transfer/receive

- 8 bytes of data at a time (4 bytes on each channel) on MHU 0 (secure)
- 8 bytes of data at a time (4 bytes on each channel) on MHU 1 (non-secure)

The APSS communicates with the following sub-system via MHU 0 and 1

- APSS and RTSS-HP sub system.
- APSS and RTSS-HE sub system.
- APSS and SE sub system.

The following Linux Kernel configurations are enabled by default to support MHUv2:

```
CONFIG_NET=y
CONFIG_UNIX=y
CONFIG_MAILBOX=y
CONFIG_ARM_MHUv2=y
CONFIG_RPMSG=y
CONFIG_RPMSG_CHAR=y
CONFIG_RPMSG_ARM=y
```

7.2 HWSEM (Hardware Semaphore)

A semaphore is a variable or abstract data type that provides a simple but useful abstraction for controlling access by multiple cores/processes to a common resource in a multi-core environment like there is in the Ensemble DevKit. The DevKit supports a total of 16 hardware semaphores. Their purpose

is to control and facilitate access to a shared resources or perform critical operations without conflicts or race conditions between two or more cores.

The following guidelines should be followed while performing a critical operations or access to a shared resource.

- Each core has to lock the HWSEM, by writing its MASTER_ID into the corresponding HWSEM base address before performing the critical operation or access to a shared resource.
- The core holding HWSEM should unlock/release the HWSEM after performing the critical operations.
- Other cores should wait/block on HWSEM if the specific HWSEM is already locked by another core.

To facilitate the above, the HWSEM driver in the Linux Kernel allows,

- Locking a specific HWSEM by writing MASTER_ID into the corresponding HWSEM base address.
- Reading current MASTER_ID of a particular HWSEM from an offset address (to know the core which locked HWSEM).
- Reading HWSEM count from an offset address (the number of times the particular HWSEM is locked).
- Unlocking HWSEM, so that other cores can lock it.

The following Linux Kernel configurations are enabled to support HWSEM:

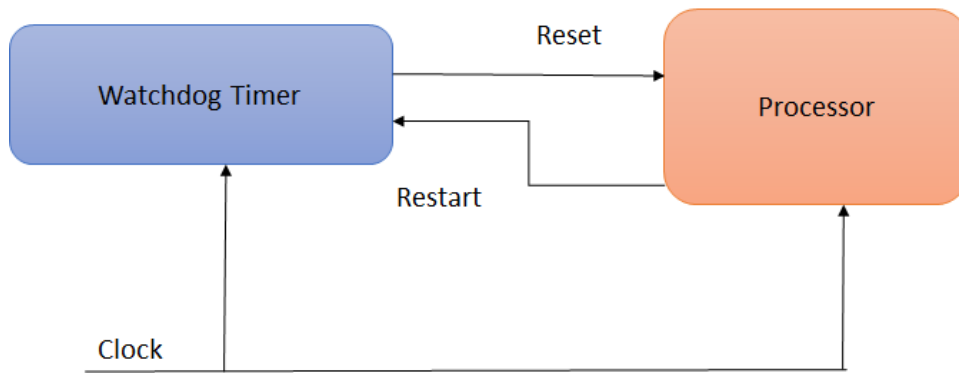
CONFIG_HWSPINLOCK=y

CONFIG_HWSEM_ALIF=y

7.3 Watchdog (SBSA Generic Watchdog timer)

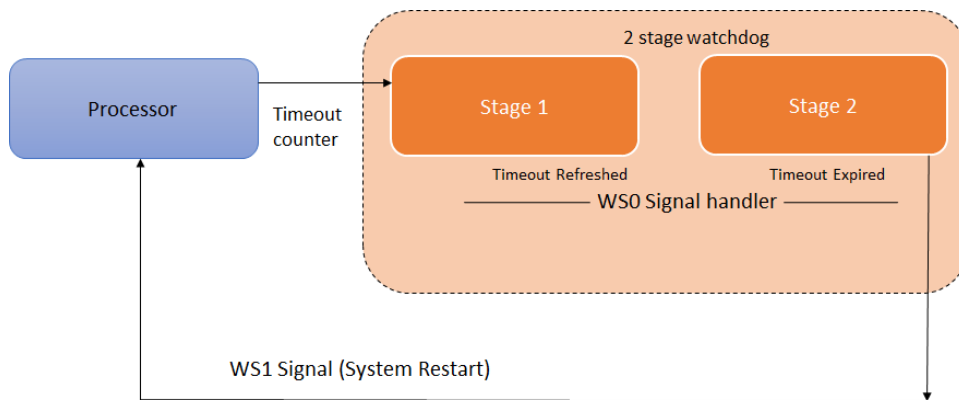
Most embedded systems need to be self-reliant. It's not usually possible to wait for someone to reboot them if the software hangs or system crashes. Some embedded designs, such as space probes, are simply not accessible to human operators. If their software ever hangs, such systems are permanently disabled. In other cases, the speed with which a human operator might reset the system would be too slow to meet the uptime requirements of the product.

A watchdog timer is a piece of hardware that can be used to automatically detect software anomalies and reset the processor if any occur. A watchdog timer is based on a counter that counts down from some initial value to zero. The embedded software selects the counter's initial value and periodically restarts it. If the counter ever reaches zero before the software restarts it or refresh the counter, the software is presumed to be malfunctioning and the processor's reset signal is asserted. The processor (and the embedded software it's running) will be restarted as if a human operator had cycled the power. Below figure shows the working of Watchdog module



The Arm SBSA Generic Watchdog has two-stage timeouts: The first signal (WS0) is for alerting the system by interrupt, The second one (WS1) is a real hardware reset that reboots the system. The working of the generic watchdog is as:

- Generic Watchdog act either as a single stage watchdog or a two stages Watchdog, it's set up by the module parameter "action".
- In the single-stage mode, when the timeout is reached, the host system will be reset by WS1. The first signal (WS0) is ignored.
- In the two stages mode, when the timeout is reached, the first signal (WS0) will trigger panic basically an interrupt will be generated and serviced by the interrupt handler. If the system is getting into trouble and cannot be reset by panic or restart properly by the kdump kernel (if supported), then the second stage (as long as the first stage) will be reached, the system will be reset by WS1 signal.



The following kernel configurations are enabled to support WDT:

```

CONFIG_WATCHDOG=y
CONFIG_WATCHDOG_CORE=y
CONFIG_WATCHDOG_HANDLE_BOOT_ENABLED=y
CONFIG_WATCHDOG_OPEN_TIMEOUT=0
CONFIG_ARM_SBSA_WATCHDOG=y
  
```

Watchdog DTS configurations:

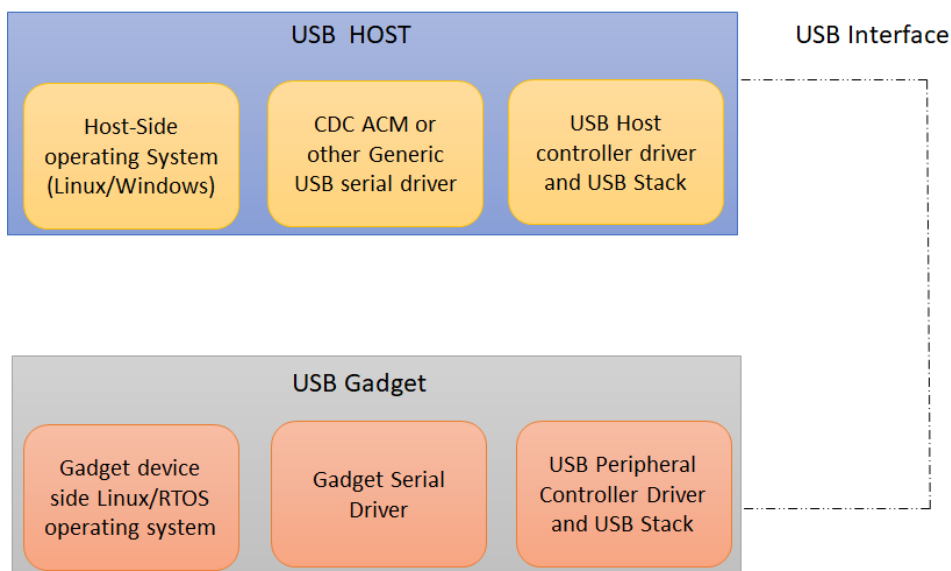
```
watchdog@1a300000 {
    compatible = "arm,sbsa-gwdt";
    reg = <0x1a300000 0x1000>,
        <0x1a310000 0x1000>;
    clocks = <&syst_pclk>;
    clock-names = "apb_pclk";
    interrupt-parent = <&gic>;
    interrupts = <0 32 4>;
    timeout-sec = <100>;
};
```

Register address 0x1a300000 is for accessing Control frame registers which gives information about Watchdog status, Watchdog offset and Watchdog compare value registers which are required for watchdog operations. Register address 0x1a310000 is for accessing Refresh frame registers which gives information about Counter refresh register which on refreshing counter restarts Watchdog operations else RESET will be generated.

7.4 USB Device mode (DWC3 Dual role controller)

Certain embedded devices have USB Device Controllers that can be used in either host mode or device mode. A USB-B micro cable allows connection to a USB host such as a PC. When used in device mode, the embedded target needs to have a 'Gadget driver' loaded which implements the attributes of the device type required. E.g., CDC ACM, CDC ECM, CDC MS devices. These devices are categorized as Composite devices with more than one functionality. User wants to configure or use USB Device as serial device gadget in that case `g_serial` gadget driver in Kernel will be required.

Below figure shows the components required in USB Host and USB Device to work like USB Serial gadget and related class component in USB Host. USB Interface is the physical connection between USB Host and USB Device using either USB-B or USB-C cable.



The DWC3 dual-role USB controller has multiple features like:

- Dual-role device capability.
- Can be configured as Host or Device.
 - Same programming model for SuperSpeed (SS), High-Speed (HS), Full-Speed (FS), and Low-Speed (LS).
- Internal DMA Controller.
- LPM protocol in USB2.0.

The DWC3 controller in Device mode exhibits features mentioned below:

- Up to 16 bidirectional endpoints, including control endpoint 0
- Software directly handles non-timing-critical and rarely occurring tasks, such as control transactions Flexible endpoint configuration
- Dynamic mappable TxFIFOs to support more Tx-endpoints than the physical FIFOs
- Simultaneous IN and OUT transfer support

DevKit device have support of DWC3 dual role controller. DWC3 can be configured either as Host or Device.

The USB controller is configured as a high-speed USB2.0 serial gadget and once connected through USB-B micro end in the target device and USB-A end on Linux Host it'll be detected as cdc-acm device with a serial device /dev/ttyACM0 on the Host machine and /dev/ttyGS0 on the target machine. As USB is a master-slave protocol all transactions will be initiated by the USB host.

Below mentioned Kernel configurations enabled for USB device support are:

```
CONFIG_USB_OHCI_LITTLE_ENDIAN=y
CONFIG_USB_SUPPORT=y
CONFIG_USB_COMMON=y
CONFIG_USB_ARCH_HAS_HCD=y
CONFIG_USB_DWC3=y
CONFIG_USB_DWC3_GADGET=y
CONFIG_USB_DWC3_ENSEMBLE=y
CONFIG_USB_GADGET=y
CONFIG_USB_GADGET_VBUS_DRAW=2
CONFIG_USB_GADGET_STORAGE_NUM_BUFFERS=2
CONFIG_USB_LIBCOMPOSITE=y
CONFIG_USB_F_ACM=y
CONFIG_USB_U_SERIAL=y
CONFIG_USB_F_SERIAL=y
CONFIG_USB_F_OBEX=y
CONFIG_USB_G_SERIAL=y
```

USB Device DTS configuration snippet: -

```
hsusb: dwc3-drd {
    usb_dual_hs: usb-dual@48200000 {
        compatible = "snps","snps,dwc3";
        reg = <0x48200000 0x100000>;
        interrupts = <GIC_SPI 26 IRQ_TYPE_LEVEL_HIGH>;
        dr_mode = "peripheral";
        maximum-speed = "high-speed";
        status = "okay";
    };
};
```

```
};
};
dr_mode = "peripheral" entry shows controller to be configured in device mode.
maximum-speed = "high-speed" entry shows controller configured as USB2.0 High speed host
```

Device detection: When the configured DevKit as USB device is connected to Linux Host using Micro-B it is detected as ttyACM0 device on Host side

```
usb 1-1: New USB device found, idVendor=0525, idProduct=a4a7, bcdDevice= 5.04
usb 1-1: New USB device strings: Mfr=1, Product=2, SerialNumber=0
usb 1-1: Product: Gadget Serial v2.4
usb 1-1: Manufacturer: Linux 5.4.25-00024-g9283a6810958-dirty with dwc3-gadget
cdc_acm 1-1:2.0: ttyACM0: USB ACM device
```

7.5 I2C ENV3-CLICK Temperature Sensor

The ENV-CLICK is a compact add-on board with a CLICK interface that contains sensors for measuring gases, pressure, humidity, and temperature. It comes with a configurable host interface that supports both SPI and I2C communication.

The ENV3 sensor with I2C Interface provides temperature readings that indicate the ambient temperature of the board. Temperature readings are communicated over a DesignWare I2C interface.

Full details on the ENV3-CLICK sensor board are available at:

<https://www.mikroe.com/environment-3-click>

Features of I2C Temperature Sensor

- DWC I2C Master driver.
- Measures temperatures from -40 degree C to +85 degree C.
- Data is read from/written to via a DesignWare I2C controller
- Thermometer accuracy is +2.0 or -2.0 deg C.

ENV3-CLICK Temperature Sensor Verification

Once the Linux is booted on the APSS, execute the following command on target to read real-time temperature from the sensor.

```
root@devkit-e7:~# cat /sys/class/i2c-adapter/i2c-0/0-0076/iio\:device0/in_temp_input
```

26.3125 Degree Celsius

Summary of the Test:

1. ENV3-CLICK temperature sensor is a client of DWC i2c(0 to 3) and using i2c_0 instance .
2. Registered interrupt for DWC_i2c is 121. Please find the cat /proc/interrupt log below.

```
root@devkit-e7:~# cat /proc/interrupts
```

```
CPU0 33: 96 GIC-0 121 Level 49010000.i2c, 49011000.i2c, 49012000.i2c, 49013000.i2c
```

3. It reads the real-time temperature value and you can observe increment/decrement in value if you read the value again after some time.
4. Sensor has configured for 16-bit resolution mode where you can read the minimum change in temperature.
5. Each time sensor is forced to into force mode and read the curent temperature value.

The following Linux Kernel configurations are enabled to support ENV3-CLICK Temperature Sensor:

```
CONFIG_I2C=y
CONFIG_I2C_BOARDINFO=y
CONFIG_I2C_COMPAT=y
CONFIG_I2C_HELPER_AUTO=y
CONFIG_I2C_DESIGNWARE_PLATFORM=y
CONFIG_I2C_DESIGNWARE_SLAVE=y
CONFIG_I2C_DESIGNWARE_CORE=y
CONFIG_IIO=y
CONFIG_HWSEM_ALIF=n
CONFIG_MAILBOX=n
CONFIG_ARM_MHUv2=n
CONFIG_RPMSG=n
CONFIG_BME680=y
CONFIG_BME680_I2C=y
CONFIG_REGMAP_I2C=y
CONFIG_CONFIGFS_FS=y
CONFIG_SYSFS=y
CONFIG_EXPERT=y
CONFIG_SYSFS_SYSCALL=y
CONFIG_SLAB=y
CONFIG_BLK_DEV_NULL_BLK=y
```

7.6 SPI ENV3-CLICK Temperature Sensor

The ENV-CLICK is a compact add-on board with a CLICK interface that contains sensors for measuring gases, pressure, humidity, and temperature. Is comes with a configurable host interface that supports both SPI and I2C communication.

The ENV3 sensor with I2C Interface provides temperature readings that indicate the ambient temperature of the board. Temperature readings are communicated over a Motorola SPI interface.

Full details on the ENV3-CLICK sensor board are available at:

<https://www.mikroe.com/environment-3-click>

Features of SPI Temperature Sensor

- DWC SSI Master driver.
- Measures temperatures from -40 degree C to +85 degree C.
- Data is read from/written to via a Motorola Serial Peripheral Interface (SPI)
- Thermometer accuracy is +2.0 or -2.0 deg C.

ENV3-CLICK Temperature Sensor Verification

Once the Linux is booted on the APSS, execute the following command on target to read real-time temperature from the sensor.

```
root@devkit-e7: ~# cat /sys/class/spi_master/spi3/spi3.0/iio\:device0/in_temp_input
26.3125 Degree Celsius
```

Summary of the Test:

1. ENV3-CLICK temperature sensor is a client of DWC SPI(0 to 3) and using SPI3 instance .
2. Registered interrupt for DWC SPI3 is 163. Please find the cat /proc/interrupt log below.

```
root@devkit-e7:~# cat /proc/interrupts
CPU0 33:    96  GIC-0 129 Level  48103000.spi, 48104000.spi, 48105000.spi, 48106000.spi
```
3. It reads the real-time temperature value and you can observe increment/decrement in value if you read the value again after some time.
4. Sensor has configured for 16-bit resolution mode where you can read the minimum change in temperature.

Each time sensor is forced to into force mode and read the current temperature value.

The following Linux Kernel configurations are enabled to support ENV3-CLICK Temperature Sensor:

```
CONFIG_SPI=y
CONFIG_SPI_MASTER=y
CONFIG_SPI_DESIGNWARE=y
CONFIG_SPI_DW_MMIO=y

CONFIG_BME680=y
CONFIG_BME680_SPI=
CONFIG_IIO=y
CONFIG_IIO_CONFIGFS=y
CONFIG_HWSEM_ALIF=n
CONFIG_MAILBOX=n
CONFIG_ARM_MHUv2=n
CONFIG_RPMSG=n
CONFIG_SLAB=y
CONFIG_BLK_DEV_NULL_BLK=y
CONFIG_CONFIGFS_FS=y
```


10.6 I2C

7.7 USB Device Detection

The USB controller is configured as high-speed USB2.0 Host and once slave device is connected on USB-A end in target device and USB-B micro end on Linux gadget (If embedded targets) or other bus powered device, The connected USB Device will be detected as CDC ACM device or Mass storage depending upon class device connected and its driver available at USB Host side. Mass storage device (Pen drive) is detected as storage device with mount points /dev/sda*, CDC ACM class device will be detected as ttyACM0 device. USB device is enumerated and functional once its class driver is loaded, So USB Host must have specific USB class drivers for device connected. E.g., if Mass storage device is connected Host must have storage class driver enabled in configurations, if CDC ACM (Serial Gadget) is connected host must have CDC ACM class driver enabled in configurations.

CDC ACM Device detection: When target device configured as USB device connected using USB-B micro with Host target

```
usb 1-1: New USB device found, idVendor=0525, idProduct=a4a7, bcdDevice= 5.04
usb 1-1: New USB device strings: Mfr=1, Product=2, SerialNumber=0
usb 1-1: Product: Gadget Serial v2.4
usb 1-1: Manufacturer: Linux 5.4.25-00024-g9283a6810958-dirty with dwc3-gadget
cdc_acm 1-1:2.0: ttyACM0: USB ACM device
```

Mass storage device detection: When USB pen drive is plugged into USB-A port of target device (Host machine)

```
usb 1-1: new high-speed USB device number 7 using xhci-hcd
usb 1-1: New USB device found, idVendor=0781, idProduct=5567, bcdDevice= 1.00
usb 1-1: New USB device strings: Mfr=1, Product=2, SerialNumber=3
usb 1-1: Product: Cruzer Blade
usb 1-1: Manufacturer: SanDisk
usb 1-1: SerialNumber: 4C530001170825116350
usb-storage 1-1:1.0: USB Mass Storage device detected
scsi host0: usb-storage 1-1:1.0
scsi 0:0:0:0: Direct-Access   SanDisk Cruzer Blade   1.00 PQ: 0 ANSI: 6
sd 0:0:0:0: [sda] 30464000 512-byte logical blocks: (15.6 GB/14.5 GiB)
sd 0:0:0:0: [sda] Write Protect is off
sd 0:0:0:0: [sda] Mode Sense: 43 00 00 00
sd 0:0:0:0: [sda] Write cache: disabled, read cache: enabled, doesn't support DPO or FUA
sda: sda1
sd 0:0:0:0: [sda] Attached SCSI removable disk
```

7.8 CDC-200

Overview

CDC is a fully Customizable Display Controller IP core family supporting the OpenWF display API specification. Several features can be configured at synthesis time and controlled at run time via a

register interface and easy to use software driver. The display controller can be applied to e.g. FPGA systems with a resource optimized, application specific feature configuration or to ASIC projects applying a more generic feature set and thus more flexibility.

Based on a highly modular architecture the CDC-Family (CDC-200,-300,-400,-500) offers a wide range of features from CDC-200 providing basic functionality with each family member offering additional configurable feature extensions up to the flag-ship CDC-500 which supports output image rotation.



7.9 CRC

Overview

Cyclic Redundancy Check (CRC) is a commonly used way to verify the integrity of a message. The CRC algorithm computes a digest for a given message following a certain computation. The computation involves dividing the given message (in its binary representation form) by a polynomial repeatedly and finding the remainder. This remainder is the CRC for the given message. There are many CRC algorithm types and they differ in the polynomial they choose (the polynomial can be of order 8, 16 or 32). The result size of the CRC algorithm depends on the polynomial chosen. The result size for CRC32, CRC16 and CRC8 algorithms are 32 bits, 16 bits and 8 bits respectively.

Implementation

The Alif CRC module supports the following 5 different CRC algorithms.

- CRC-8-CCITT
- CRC-16
- CRC16-CCITT
- CRC-32
- CRC-32C

The Alif CRC module outputs the CRC for an input data based on the values set in the below control registers.

1. Init register - Configures the required algorithm type.
2. Seed register - Writes the seed value.
3. Data registers - Writes the input data either 4 bytes at a time (for CRC32) or 1 byte at a time (for CRC8 and CRC16).

Once all data has been written, the CRC output/result read from the result register.

The register address information can be found in the CRC hardware reference document.

The Alif CRC module functionality is used for the APSS using Linux kernel device driver. The driver registers with the Linux crypto subsystem for all the above algorithms.

The following Linux Kernel configurations must be enabled in the Linux kernel defconfig to enable Alif CRC Linux device driver in the Linux kernel (xiplmage).

```
CONFIG_INET=y
CONFIG_CCRYPTO=y
CONFIG_CCRYPTO_MANAGER=y
CONFIG_CCRYPTO_USER=y
CONFIG_CCRYPTO_CRC32C=y
CONFIG_CCRYPTO_CRC32=y
CONFIG_CCRYPTO_USER_API_HASH=y
CONFIG_CCRYPTO_HW=y
CONFIG_CCRYPTO_DEV_ALIF_CRC=y
```

Adding "apss-crc" into DISTRO_FEATURES variable within conf/local.conf file of the APSS Linux project builds xiplmage with the above configs enabled and cramfs-xip containing CRC test program (/opt/linux_dd_test/crc/test_crc).

Verification

The Alif CRC Linux device driver can be tested with the test program /opt/linux_dd_test/crc/test_crc (available in cramfs-xip upon adding 'apss-crc' into DISTRO_FEATURES). This program uses the Socket interface and AF_ALG socket family to talk to the Alif CRC driver. This program takes the CRC type and text as an input and provides the CRC as the result of the given input. The results of the program can then be verified with a standard [online CRC calculator](#).

Usage: /opt/linux_dd_test/crc/test_crc <crc_type> <input_string>

valid crc types: crc32, crc32c, crc16, crc16-ccitt, crc8 crc32-alif-linux crc32-linux crc32c-linux

Examples:

```
root@devkit-e7:~# test_crc crc32 starlight
```

```
Init
```

```
Update
```

```
result : 40437e6c
```

```
input: starlight
```

```
crc type: crc32
```

```
Result is: c1 2e 34 a5
```

```
root@devkit-e7:~# test_crc crc32c starlight
```

```
Init values
```

```
Init
```

```
Update
```

```
result : 564e87a0
```

```
input: starlight
```

```
crc type: crc32c
```

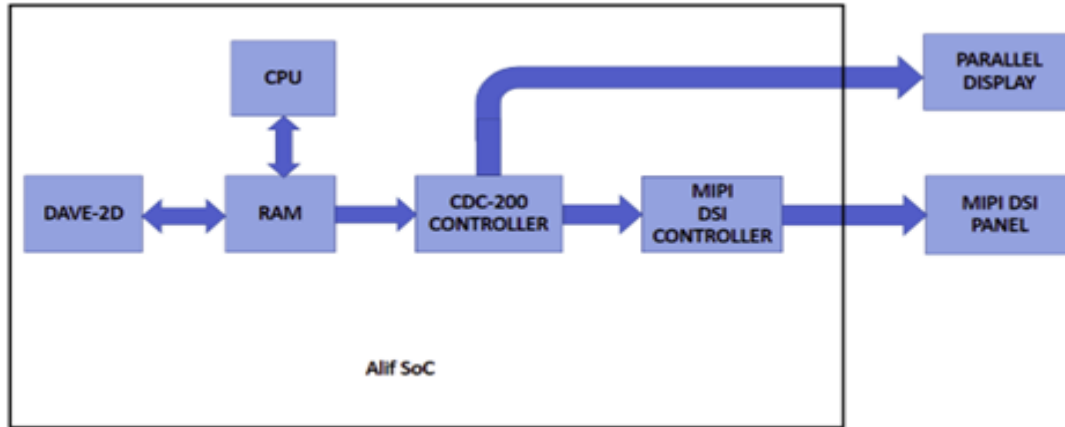
```
Result is: 46 62 6b 62
```

7.10 D/AVE-2D

Overview

D/AVE-2D is an IP developed for sophisticated vector-based graphics applications. D/AVE is perfectly suited for embedded systems and offers a wide range of graphics primitives. It is a 2D GPU Hardware IP optimized for smaller FPGA/ASICs for providing cost efficient high-quality vector-graphics with sub-pixel processing and extended anti-aliasing. Use-cases for the same include – GUI (Graphical User Interface), Navigation, Dashboard and On-Screen Display for consumers, wearables, industrial, automotive and aviation.

The CPU co-ordinates with different graphics sub-system blocks. The D/AVE-2D performs rendering operations, and the CDC200 forwards the rendered buffer to the Display connected. D/AVE-2D is an internal peripheral and works directly with the memory. It takes a memory buffer in SRAM, renders to it and writes back to the SRAM



8 Linux device drivers RAM usage

The following table highlights the size of xiplmage, SRAM0 used when a specific Linux device driver is enabled. The total size of SRAM0 is 4096 KB.

Linux Device Drivers	Xiplmage size (in MB)	RAM used/ SRAM0 used (in KB)	RAM Free/SRAM0 Free (in KB)
Basic Drivers: UART-2B + Pin-Mux + RTC + WDT	1.6	1692	1492
Basic Drivers + USB	1.8	2556	596
Basic Drivers + USB +SPI	1.8	2644	512
Basic Drivers + USB + SPI + MHU + HWSEM	2.1	2796	356
Basic Driver+ SD Shared	2.3	2228	304
Basic Driver+SD Boot	2.3	2104	356

9 Linux Kernel Feature

The following Linux Kernel features are enabled by default in the Linux kernel configuration file DevKit_e7_defconfig.

Kernel Feature	Enabled in the default Linux kernel config	Enabled with DISTRO_FEATURES	DISTRO_FEATURES value
Arm System Type (ARMv7-A)	Yes	Yes	
Arm Uniprocessor	Yes	Yes	
Arm SMP	No	No	
Tick Based CPU Accounting	Yes	Yes	
PREEMPT_NONE	Yes	Yes	
SLUB	Yes	Yes	
SLAB_CACHE_MERGE	Yes	Yes	
VMSPLIT_3G	Yes	Yes	
THUMB2 – KERNEL	Yes	Yes	
VFP	Yes	Yes	
NEON	Yes	Yes	
KERNEL MODE NEON	Yes	Yes	
FILE SYSTEM			
CRAMFS	Yes	Yes	
SYSFS	No	Yes	apss-spi or apss-usb
INOTIFY & DNOTIFY	Yes	Yes	
DRIVERS			
DWC - UART2-B	Yes	Yes	
PINMUX	Yes	Yes	

RTC	Yes	Yes	
DWC SPI	No	Yes	apss-spi
DS1722 – Temp Sensor (SPI)	No	Yes	apss-spi
WATCH-DOG – ARM SBSA	Yes	Yes	
DWC3 USB	No	Yes	apss-usb
USB GADGET – CDC-ACM	No	Yes	apss-usb
MTD	Yes	Yes	
BLOCK Device	Yes	Yes	
MHU	No	No	apss-mhu
HWSEM	No	Yes	apss-hwsem
USB Host – Mass storage/CDC ACM class	No	Yes	apss-usb
SDHCI boot	No	Yes	apss-sd-boot
SDHCI mount	Yes	Yes	apss-sd-share
ETHERNET	No	Yes	apss-ethernet
I2C-I3C	No	Yes	apss-i2c-i3c
I2S	No	Yes	apss-i2s
CDC-200	No	Yes	apss-cdc200/MIPI-DSI
Utimer	No	Yes	apss-utimer
I3C	No	Yes	apss-i3c
CRC	No	Yes	apss-crc
DAC	No	Yes	apss-dac12

10 Reference

- Please refer [Yocto reference manual](#) and [bitbake reference manual](#) on how to create a recipe, compile, and add it to the file system.
- Renesas APPLICATION NOTE [XIP Linux for RZ/A1](#).
- [Cramfs-tools](#)
- [Application XIP Instructions For OMAP](#)

11 Document History

Version	Change Log
0.5.2_3	Support for v0.5.2 release with two sets of pre-built images